Interview set 4

1. How does Python handle memory management?

1. Private Heap Space

- All Python objects and data structures are stored in a **private heap**.
- This heap is managed internally by the Python interpreter.
- You **cannot directly access or manipulate memory addresses**.

2. Reference Counting (Core Mechanism)

- **Every object in Python has a reference count.**
- **This count tracks how many variables reference the object.**

```
a = [1, 2, 3]
b = a   # reference count increases
del a   # reference count decreases
```

- When the reference count becomes **zero**, Python automatically deallocates the memory.
  Fast and efficient, but has limitations with circular references.

3. Garbage Collection (Cycle Detection)

- Python uses a **cyclic garbage collector** to handle **circular references** (objects referencing each other).

```
import gc

class A:
    def __init__(self):
        self.ref = None

obj1 = A()
obj2 = A()

obj1.ref = obj2
```

```
obj2.ref = obj1

del obj1
del obj2

gc.collect()  # cleans cyclic references
```

The gc module helps in:

- Manual garbage collection
- Debugging memory leaks
- Controlling GC behavior

4. Memory Pools & Allocation (Efficient Handling)

Python uses a specialized allocator called:

- **PyMalloc**

It optimizes memory usage by:

- Allocating memory in **blocks (arenas → pools → blocks)**
- Reusing freed memory instead of returning it to OS immediately

Hierarchy:

- **Arena (256 KB)**
  - Pool (4 KB)
    - Block (small objects)

👉 This improves performance for small object allocation.

5. Generational Garbage Collection

Objects are categorized into **generations** based on lifespan:

Generation	Description
Gen 0	New objects
Gen 1	Survived one GC cycle

Generation	Description
Gen 2	Long-lived objects

- Frequent cleanup in Gen 0
- Less frequent in higher generations

👉 Optimization assumption:

“Most objects die young”

6. Memory Deallocation Behavior

- Python **does not always return memory to OS immediately**
- It keeps memory for reuse (performance optimization)

7. Important Interview Points 🚀

- Python uses:
  - ✅ Reference Counting (primary)
  - ✅ Garbage Collector (for cycles)
- Memory is managed in a **private heap**
- Uses **PyMalloc** for efficient allocation
- Supports **generational GC**
- del **does NOT delete object**, it only removes reference
- Circular references require **GC, not ref counting**

Answer for Interview:

Python manages memory automatically using reference counting and garbage collection, where objects are deleted when their reference count becomes zero, and cyclic references are handled by the garbage collector.”

2. What is the difference between multiprocessing vs multithreading?

Multiprocessing vs Multithreading (Simple Explanation)

Both are used to run multiple tasks at the same time, but they work differently.

---

1. Multithreading (Lightweight)

Multiple threads inside the same process

- **Share same memory**
- **Fast to create**
- **Good for I/O tasks (API calls, file reading)**
- **One kitchen , multiple chefs working together**

2. Multiprocessing (Heavy but Powerful)

- **Multiple separate processes**
- **Each process has its own memory**
- **Slower to create**
- **Best for CPU-heavy tasks (calculations, ML)**
- **Multiple kitchens , each with its own chef**

One-Line Interview Answer ⭐

"Multithreading uses shared memory and is suitable for I/O-bound tasks but is limited by the GIL, whereas multiprocessing uses separate memory spaces and enables true parallel execution, making it ideal for CPU-bound tasks."

3. Explain GIL (Global Interpreter Lock)

"GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, limiting multithreading performance for CPU-bound tasks but simplifying memory management."

4. What are generators and why are they useful in data engineering?

Generators are special functions that return data one by one, instead of returning everything at once.

They use the keyword yield instead of return.

5. Difference between iterator vs generator

__iter__() vs yield

Iterator is an object that lets you loop over data one by one

It must have:

__iter__() → returns iterator object

__next__() → gives next value

Generator is an easy way to create an iterator using yield

Key Difference

Feature	**Iterator**	**Generator**
Creation	Class-based	Function-based
Complexity	More code	Less code
Methods	Need __iter__() & __next__()	Uses yield
Readability	Complex	Simple
Use Case	Custom logic control	Data pipelines

One-Line Interview Answer ⭐

"An iterator is an object implementing __iter__() and __next__() for manual iteration control, whereas a generator is a simpler way to create iterators using yield, enabling lazy and memory-efficient data processing."

6. How do you handle large file processing efficiently?

- Chunking
- Streaming
- Generators